Dipartimento di Ingegneria e Scienza dell'Informazione

Corso di Laurea in
Information, Communication and Electronics Engineering

Elaborato finale

Tiny Acceleration of Liquid Foundation Models

Selective FPGA Offloading of the SwiGLU Feed-Forward Block of LFM2.5-1.2B on the Kria KV260

Relatore
Roberto Passerone

Laureando
Jacopo Marchesin

Correlatore
Danilo Pau (STMicroelectronics)

Anno Accademico 2025/2026

Acknowledgements

...thanks to...

Contents

Abstract

Conversational artificial intelligence on edge devices must satisfy three constraints at once: a per-token generation latency low enough for natural spoken dialogue, a single-digit-watt power envelope, and local-only execution for privacy. Meeting these constraints with a modern transformer-based language model on hardware with limited resources is non-trivial, because such models require significant compute, memory, and power.

This thesis investigates whether a combination of aggressive quantization and selective hardware acceleration can bring a competitive conversational model within the edge envelope. The target model is the 1.2-billion-parameter language backbone of Liquid AI's LFM2.5-Audio, an end-to-end speech model co-designed for efficient CPU inference. Fine-grained profiling of the model on a quad-core Arm Cortex-A53 shows that a single kernel, the MatMul + SwiGLU feed-forward block, accounts for about 68% of the per-token runtime. This result motivates a heterogeneous design in which only that dominant kernel is offloaded to a custom FPGA accelerator on the AMD-Xilinx Kria KV260, while attention, normalization, the gated short convolution, and the final projection remain on the CPU.

The accelerator is implemented in Vitis HLS as a five-stage dataflow pipeline operating on 4-bit (Q4_K) weights and 8-bit activations, and is integrated into the `llama.cpp/ggml` inference stack through a fused hardware operator with permanent per-layer weight caching, interrupt-driven synchronization, and transparent fallback to CPU execution. The design is analysed at stage granularity, showing that performance is bound by off-chip memory bandwidth rather than arithmetic capacity, and that it operates near the device's logic and routing limits (87% LUT occupancy).

The work includes the model profiling, the software integration in `llama.cpp`, the FPGA accelerator design and characterization, a throughput/efficiency study across CPU thread counts (T1–T4), and a comparison against an STM32MP257F-DK edge MPU. The accelerated path improves performance-per-watt by up to 57% (prefill) and 44% (decode) at low thread counts and enables sub-500 ms decode latency with only two active CPU threads, while the analysis quantifies the memory-bandwidth, orchestration, and routing constraints that bound the attainable throughput.

1 Introduction and Background

1.1 Introduction

The rapid growth of intelligent sensing devices is shifting AI from power-hungry data centers and supercomputing systems to low-power edge platforms, where perceptual AI, text generation, reasoning, and control occur near the data source under challenging latency, energy, and privacy constraints [30].

Within the domain of perceptual AI, TinyML was the initial step toward edge intelligence. It has been demonstrated that inference workloads are feasible on microcontrollers (MCUs) using moderately parameterized CNN and RNN models [1, 2]. Since 2024, the edge has evolved to embrace Generative Edge AI. Modern Edge AI is built on heterogeneously quantized Transformer architectures, including small language models (SLMs), which seek to retain useful generative and reasoning capabilities under tight resource (memory and computation) budgets [20, 17]. Techniques such as deep quantization, pruning, knowledge distillation, and hardware-aware neural architecture design have become central to reconciling model accuracy with edge deployability [15]. Delivering real-time responsiveness requires tight integration of models, algorithms, and low-power hardware.

In this landscape, conversational audio is a natural target for edge deployment: its pipeline decomposes into independently optimizable modular stages, its temporal features operate at comparatively low dimensionality, and its biometric content makes local processing preferable from both privacy and regulatory perspectives. Conversational systems impose stringent real-time constraints that benefit from the elimination of network latency, and their modular structure allows individual stages to be aggressively quantized without prohibitive quality loss.

This thesis first defines the objectives of the proposed work on conversational systems for edge devices in Section 1.2. Section 1.3 provides an overview of the state of the art in modern AI architectures, edge AI systems, and embedded speech processing. Section 1.4 outlines the specific findings and contributions of this work. The methodology for achieving conversational-quality inference under strict compute and memory budgets is detailed in Chapter 2, while Chapter 3 presents the experimental setup, the evaluation, and the discussion of the broader implications and limitations of the findings. Chapter 4 concludes the work and outlines future directions.

1.2 Research Question

Conversational AI systems aim to provide rich, context-aware interactions through natural spoken dialogue. Achieving this capability requires models that can process audio input, understand and reason upon user intentions, and generate appropriate responses within strict real-time constraints. At the same time, conversational audio often contains sensitive and biometric information, making local processing desirable from both privacy and reliability perspectives.

Conversational speech interfaces operate within defined latency thresholds [9, 7]: response times of 100–200 ms are perceived as instantaneous, delays of 0.5–1 s are categorized as immediate and remain acceptable for sustained dialogue, and intervals exceeding 1 s degrade the perceived interaction quality. Natural human turn-taking averages approximately 230 ms [7], placing the acceptable machine-response boundary near the transition between the instantaneous and immediate bands. This leaves a narrow window for the entire pipeline (audio encoding, ASR, language modeling, and text-to-speech synthesis) to complete on a per-utterance basis.

Edge devices operate under strict limits in compute capability, memory footprint, and energy consumption, typically drawing between 3 and 10 W of power for embedded platforms in the smart-assistant and IoT category. Yet meaningful conversational interaction relies on resource-intensive transformer-based language models whose autoregressive decoding generates one token at a time, with each token requiring tens to hundreds of billions of multiply-accumulate operations [27].

A viable edge deployment must therefore satisfy three constraints simultaneously:

(i) per-token generation latency within the 200–500 ms window such that the aggregated pipeline remains under the end-to-end budget,

(ii) total system power in the single-digit-watt range compatible with embedded and battery-backed operation,

(iii) local-only execution that preserves the privacy of conversational data without reliance on cloud offload.

The central question this work poses is whether a combination of model quantization and selective hardware acceleration can bring a competitive conversational model within these constraints on a resource-limited edge platform.

1.3 State of the Art

1.3.1 Evolution of Modern AI Models

Recent years have seen rapid progress in AI driven by hyper-parameterized neural topologies trained on tens of trillion-token datasets. Transformer-based models have become the dominant paradigm across multiple modalities [29]. In vision, transformer architectures and diffusion models have enabled high-capacity systems to achieve image understanding and generation [14]. Large language models (LLMs) have demonstrated strong reasoning, chain-of-thought (CoT) capabilities, dialogue, and code generation [19]. Similarly, research on speech has increasingly adopted end-to-end neural architectures that unify acoustic modeling, language modeling, and decoding within a single framework [11]. As these models become more capable, they are increasingly integrated into systems that interact with users and the physical world in real time.

1.3.2 Real-time and Edge AI Systems

Recent research has focused on integrating AI models into systems that respond to continuous streams of sensory input. Vision–Language–Action (VLA) architectures exemplify this trend by integrating visual-audio-tactile perception with language-based reasoning and control policies [16, 18, 5]. Applications such as conversational assistants, robotics, and augmented interfaces require real-time inference with strict latency constraints and reliable operation in time-varying environments. These requirements motivated increased interest in edge-based inference, where processing occurs close to the sensing element, eliminating cloud delay concerns. Several methods have therefore been proposed to deploy machine learning models on the edge, operating under limited compute, memory, and energy budgets. From the early work on TinyML, more recent approaches harness model compression techniques such as quantization, pruning, and knowledge distillation, as well as hardware-aware neural architecture design [12]. Together, these advances form the foundation of modern Edge AI, which seeks to balance model capability with the strict resource constraints of edge platforms.

1.3.3 Speech and Conversational Architectures

De-facto standard speech processing follows a modular pipeline in which an ASR module converts audio to text, a natural language understanding component extracts intent, a language model generates a textual response, and a TTS engine synthesizes the output waveform. This decomposition enables each stage to be optimized independently for latency, accuracy, and power, and is the dominant architecture in deployed conversational systems. Typical ASR models range from compact encoder-only architectures (Whisper-tiny at 39 M parameters) to large encoder-decoder designs (Whisper-large-v3 at 1.55 B parameters) [28], while TTS engines span 10 M to 500 M parameters. Streaming ASR models allow speech to be transcribed incrementally as audio is received, reducing the time to first token (TTFT) compared to batch-based processing. The aggregate parameter count across three or four separate models can reach several billion, and the per-stage inference latencies sum within the end-to-end response budget of 200–500 ms.

Recent work has explored end-to-end architectures that unify these stages into a single neural model. LFM2-Audio [4] represents this trend well: a 1.5 B-parameter model that jointly handles audio encoding, language modeling, and speech synthesis within one framework, eliminating inter-stage

latency and easing the deployment footprint. LFM2.5 denotes a subsequent release of the same architecture with updated weights; the 1.2 B language backbone used in this work is architecturally identical across both releases. The deployment requirements of such models vary substantially; a quantitative profiling of representative checkpoints, presented in Section 1.3.5, motivates the specific model choice adopted in this work. While end-to-end architectures simplify the system design, their computational cost remains concentrated in a single large model rather than distributed across independently scalable stages, making hardware acceleration of the dominant internal kernel an attractive optimization target.

1.3.4 Edge Speech Systems and On-Device Conversational AI

The characteristics of speech signals make audio workloads well suited for edge deployment. Lightweight components such as wake-word detection and voice activity detection can run continuously on MCUs burning power under 1 W, while more demanding pipeline stages (ASR, language modeling) are typically hosted on single-board computers or heterogeneous SoCs in the 3–15 W range. The Raspberry Pi 4B (quad-core Cortex-A72 at 1.5 GHz, up to 8 GB RAM) has emerged as a common evaluation platform for on-device speech systems, hosting engines such as Faster-Whisper for real-time voice interaction [6]. At the lower end of the resource spectrum, comparative studies of phoneme-based and word-based recognition across MCU-class devices (Cortex-M4/M7 at 64–480 MHz, with hundreds of kilobytes of SRAM) identify specific architectures that optimize the accuracy-versus-memory trade-off for offline keyword spotting [22]. Prototype systems integrating Vosk or Whisper-based ASR with local NLP microservices have demonstrated sub-second speech-to-text latency entirely on edge hardware [8], validating the feasibility of local-only processing for conversational audio.

Despite this progress, existing works have largely addressed individual pipeline components in isolation. Comparatively little research examines conversational interaction as an end-to-end systems problem in which the language backbone, the ASR front-end, and the TTS output stage must all complete within the latency and power budget defined in Section 1.2. Addressing this challenge requires not only lightweight models but also system-level optimization spanning streaming inference, hardware-software partitioning, and selective acceleration of the dominant computational kernels.

1.3.5 Profiling of Speech Models for Edge Deployment

To evaluate the feasibility of conversational inference on edge devices, a representative set of speech and multimodal models was profiled using gguf-parser-go[1], a tool that extracts structural and resource information from GGUF-format checkpoints. The profiling covers parameter count, memory footprint, memory bandwidth requirements, and estimated compute throughput. These are the same metrics used in a recent large-scale study of 21,039 checkpoints across six workload categories [21], which found that audio-language models define the lightest deployment envelope (mean 13.41 GB/s bandwidth, 134 GFLOPS compute, 826 MiB UMA). One observation was that parameter count alone is an insufficient predictor of edge feasibility. The resulting comparison, illustrated in Fig. 1.1, reveals wide variation in deployment requirements: parameter counts span from 500 M (OuteTTS-0.2-500M-Q4) to 9.4 B (THUDM_GLM-4-9B-0414-Q4), RAM footprints range from 0.29 GB (LFM2.5-Audio-1.5B-Q4, the lowest among all profiled models) to 2.67 GB (Qwen2-7B-LLM-Q4), and estimated memory bandwidth demand varies from 3.6 GB/s to 65 GB/s. Several architectures, including the 7–9 B parameter class, exceed practical memory and bandwidth limits for embedded platforms despite offering strong conversational capabilities. In contrast, LFM2.5-Audio-1.5B[2] combines the lowest RAM footprint with moderate bandwidth requirements (3.9 GB/s) and a competitive parameter scale (1.18 B), positioning it closest to the desired resource envelope of embedded conversational devices.

Based on this analysis, LFM2.5 was chosen as the primary candidate for the subsequent hardware and software studies, as it lies closest to the efficiency–capability trade-off required for conversational inference under edge resource constraints.

[1] `https://github.com/gpustack/gguf-parser-go`

[2] `https://huggingface.co/LiquidAI/LFM2.5-Audio-1.5B`

Figure 1.1: Model throughput (TOPS) as a function of resource requirements. Models with the highest throughput at the lowest resource cost appear toward the bottom left.

1.3.6 Related Work on the Kria KV260 Platform

Following the identification of LFM2.5 as a suitable candidate for edge deployment, the next step was to map its computational workload onto an appropriate hardware platform. Field-Programmable Gate Arrays (FPGAs) provide a compelling and flexible target for edge AI acceleration due to their ability to exploit fine-grained parallelism, customize dataflow, and operate within tight power and latency budgets.

Recent work on the AMD-Xilinx Kria KV260 demonstrated the feasibility of accelerating transformer models. The three designs surveyed below target both feed-forward network (FFN) and self-attention matrix multiplications, which together accounted for the greater part of the inference latency and bandwidth consumption in transformer workloads. A 6.6k-parameter Tiny Transformer for ECG arrhythmia detection deployed on the KV260 achieved 56.62 ms total inference latency at 77 MHz, with the FFN contributing 54.9% of runtime (31.11 ms) [10]. A tiled matrix multiplication accelerator targeting the Q, K, and V self-attention projections of DistilBERT achieved a 7× speedup

over the Arm CPU baseline at 100 MHz and 3.3 W, delivering 3.12 GFLOPS of standalone GEMM throughput on the KV260 [25]. A BRAM-banked double-buffered matrix multiplication accelerator on the same platform reached a peak compute throughput of 22.83 GOPS for a DistilBERT-scale FFN multiplication (7.31× over its baseline), completing the operation in 13.2 ms at 100 MHz. The study identified PS-side data handling as the dominant system-level bottleneck, a finding consistent with the orchestration overheads observed in the present work (Section 3.5) [33].

Existing approaches generally focus on accelerating full transformer blocks or large-scale models, where attention and feed-forward computations are treated together as co-dominant workloads. This work targets the minimal set of computational kernels required to achieve conversational-level inference, focusing on feed-forward acceleration. The profiling results presented in Section 2.2 show that, for small models such as LFM2.5-1.2B, the execution cost is overwhelmingly dominated by feed-forward components, with attention contributing a comparatively smaller fraction of the total runtime. This shift in computational balance motivated a different design perspective, in which the feed-forward path becomes the primary target for acceleration. Furthermore, the LFM2.5-1.2B variant operates using quantized weights without bias terms, simplifying the arithmetic structure of the model and making it particularly well suited for efficient FPGA implementation.

1.4 Contributions

This work provides five contributions.

1. **System-level bottleneck identification for edge conversational inference.** Fine-grained profiling of LFM2.5-1.2B on an Arm-based edge platform shows that feed-forward MatMul + SwiGLU computation dominates runtime, motivating selective acceleration instead of full-model hardware mapping.

2. **A practical CPU–FPGA offload integration in the llama.cpp/ggml stack.** A fused FFN hardware operator is implemented with graph-level interception, layer-aware weight caching, interrupt-driven synchronization, and fallback compatibility with standard CPU execution.

3. **A resource-aware FPGA design and characterization.** The accelerator is implemented as a staged dataflow pipeline and analyzed at stage granularity, highlighting where execution is limited by memory transfer across all projection stages, including the down-projection. The design operates near device limits (e.g., high LUT occupancy), exposing realistic scaling constraints on KV260-class platforms.

4. **Quantitative efficiency/performance trade-off across thread regimes.** CPU Only versus FPGA+CPU measurements across thread counts (T1–T4) are reported using throughput, latency, power, and performance per watt. Results show clear efficiency benefits at low and moderate CPU thread counts, while high-thread regimes remain constrained by data movement and orchestration overheads.

5. **Comparative analysis against an edge MPU.** The STM32MP2, a low-power dual-core Arm A35 running at 1.5 GHz, is benchmarked against the FPGA+CPU implementation in terms of latency and energy consumption.

2 System Design and FPGA Accelerator

This chapter describes the proposed methodology. Section 2.1 first introduces the two architectural notions on which the rest of the chapter relies: the SwiGLU feed-forward block and the LFM2 hybrid backbone. Section 2.2 then identifies the dominant computational kernel through fine-grained profiling, Section 2.3 details the CPU–FPGA orchestration that offloads that kernel, and Section 2.4 presents the design of the FPGA accelerator itself.

2.1 Background

2.1.1 The Feed-Forward Block and SwiGLU

Modern transformer language models alternate two kinds of sub-layer within each block: a token-mixing operator (self-attention or, in LFM2, a short convolution) and a position-wise feed-forward network (FFN). The FFN is applied independently to every token's hidden vector and is responsible for most of a model's parameters and arithmetic. A classic FFN expands the d-dimensional hidden vector to a larger intermediate dimension d_{ff} through one linear projection, applies a pointwise nonlinearity, and projects back to d through a second projection. Gated Linear Unit (GLU) variants replace the single expansion with two parallel projections combined by an element-wise gate, which improves quality at an equal parameter budget and has become standard in recent large language models. LFM2 uses the SwiGLU variant.

SwiGLU combines a SiLU-gated branch with a linear branch. The SiLU (also called Swish) activation is $\text{SiLU}(z) = z\,\sigma(z)$, where σ is the logistic sigmoid; it is a smooth, non-monotonic function that acts as a soft gate, passing large positive activations while suppressing negative ones. Given the input activation $x \in \mathbb{R}^d$ (here $d = 2048$), SwiGLU computes a gate projection $X_1 = xW^T$ and an up projection $X_2 = xV^T$, both mapping to the intermediate dimension d_{ff} (here 8192); it applies SiLU to the gate, multiplies the two branches element-wise, and projects the result back to d with the down projection W_{down}:

$$\text{FFN}(x) = (\text{SiLU}(xW^T) \odot (xV^T))W_{down}^T \tag{2.1}$$

where $\odot$ denotes element-wise multiplication. LFM2 applies no bias terms in the FFN, which simplifies the arithmetic and is exploited by the hardware design in Section 2.4.

The cost of this expressiveness is that SwiGLU uses three large weight matrices (W and V of size $d \times d_{\text{ff}}$ and W_{down} of size $d_{\text{ff}} \times d$) rather than the two of a classic FFN. At $d = 2048$ and $d_{\text{ff}} = 8192$ each matrix holds $2048 \times 8192 \approx 16.8\,\text{M}$ weights, so a single block's FFN contains roughly 50 M parameters and dominates both the storage footprint and the multiply-accumulate count of the block. This is what makes the feed-forward path the natural acceleration target identified in Section 2.2.

2.1.2 The LFM2 Hybrid Backbone

LFM2 is a family of edge-first language models from Liquid AI, designed through a hardware-in-the-loop architecture search that targets CPU latency and memory directly rather than parameter count alone [4]. The resulting backbone is a minimal hybrid: most blocks use an inexpensive gated short convolution for token mixing, and only a small minority use grouped-query attention (GQA) [**?**] to recover long-range modeling. This combination delivers up to roughly 2× faster prefill and decode on CPUs than similarly sized attention-only models, which is what makes LFM2 a credible starting point for edge deployment.

The 1.2-billion-parameter backbone used in this work has the hyperparameters listed in Table 2.1. It comprises 16 transformer blocks over a hidden dimension of 2048, of which 10 are gated short-convolution blocks and 6 are GQA blocks; every block, regardless of its mixer, contains a SwiGLU

FFN that expands to a feed-forward dimension of 8192.

Table 2.1: Hyperparameters of the LFM2-1.2B language backbone.

Hyperparameter	**Value**
Total parameters	1.2 B
Transformer blocks	16
Hidden dimension (d_{model})	2048
Feed-forward dimension (d_{ff})	8192
Gated short-convolution blocks	10
Grouped-query attention blocks	6
Attention heads / KV groups / head dim	32 / 8 / 64
Convolution kernel size (k)	3
Normalization	pre-norm RMSNorm
Positional encoding	RoPE (attention blocks)

Internally, each block follows the pre-norm residual structure illustrated in Figure 2.1: the hidden state is normalized with RMSNorm, passed through the sequence mixer, and added back to the residual stream; it is then normalized again, passed through the SwiGLU FFN, and added back. The sequence mixer is either a gated short convolution (a depthwise causal convolution of kernel size $k = 3$ surrounded by input-dependent multiplicative gating, which provides cheap local mixing with favourable cache behaviour on CPUs) or grouped-query attention, which shares key and value projections across groups of query heads (here 32 query heads over 8 key/value groups) to reduce key–value cache traffic, augmented with rotary position embeddings and query–key normalization.

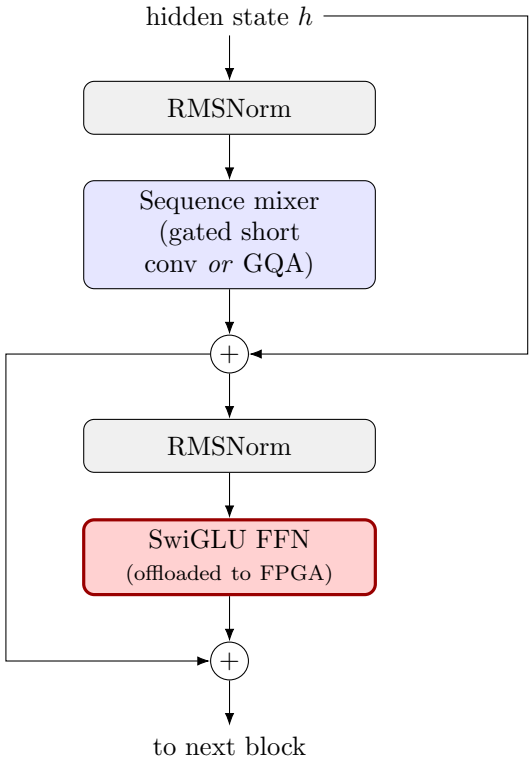

Figure 2.1: Structure of a single LFM2 transformer block. Each of the 16 blocks applies a sequence mixer (a gated short convolution in 10 blocks, grouped-query attention in 6) followed by a SwiGLU feed-forward network, both wrapped in pre-norm residual connections. Only the SwiGLU FFN is offloaded to the FPGA.

Two properties of this structure shape the rest of the chapter. First, because a SwiGLU FFN appears in all 16 blocks, the feed-forward path aggregates to the single largest share of per-token computation, as the profiling in Section 2.2 confirms. Second, the FFNs are never adjacent: consecutive

feed-forward blocks are always separated by a full sequence mixer and two normalization and residual steps. In the heterogeneous design of Section 2.3, those intervening operators remain on the CPU, so the accelerator cannot chain one FFN directly into the next; instead, every generated token incurs sixteen separate CPU–FPGA handoffs, one per block.

2.2 Profiling under CPU Execution

To address the research question of enabling conversational-quality inference within the constraints of edge devices, this work adopts a systems-driven, top-down methodology that combines model profiling with targeted hardware acceleration. Rather than attempting to accelerate the entire conversational pipeline, the approach focuses on identifying the computational bottlenecks of the language backbone that powers the selected speech model and mapping the most demanding operations to dedicated hardware.

The starting point of the approach is the LFM2-Audio architecture, whose conversational capabilities are supported by a 1.2 B-parameter language backbone. To understand its execution characteristics on embedded hardware, the inference engine implemented by llama.cpp was modified to instrument the execution of individual architectural blocks. This profiling allows the collection of fine-grained timing information for each major component of the model during inference.

Table 2.2: LFM2.5-1.2B CPU execution profile.

Architectural Block	**Time (μs)**	**% of Total**
MatMul + SwiGLU	10,136,836	68.08%
Final Linear Projection	1,631,552	10.96%
Gated Short Convolution	1,555,396	10.44%
GQA Attention (MatMul)	1,318,155	8.85%
Unmapped Nodes	174,103	1.17%
Memory Ops & KV Cache	28,330	0.19%
RMSNorm / LayerNorm	27,328	0.18%
Residual Connections & Addition	11,446	0.08%
Input Embedding Lookup	4,006	0.03%
Positional Encodings (RoPE)	1,840	0.01%

Table 2.2 summarizes the resulting execution profile on a quad-core Arm® Cortex®-A53 platform. These results indicate that the performance of the model is primarily determined by a small set of dense linear algebra operations. In particular, the feed-forward MatMul + SwiGLU blocks represent the principal computational bottleneck, followed by the projection layers and attention-related matrix multiplications. Since these operations exhibit regular and predictable memory access patterns, they are well suited for hardware acceleration via spatial architectures that can exploit custom dataflow and data reuse.

Amdahl's Law [3] bounds the theoretical benefit of this strategy: with the feed-forward MatMul + SwiGLU block (Eq. (2.1)) accounting for approximately 68% of total execution time, the maximum achievable system speedup is limited to $1/(1-0.68) \approx 3.1\times$, even if the accelerated portion were to execute instantaneously. In practice, the non-FFN overhead was independently measured at approximately 126 ms per token by substituting a pass-through IP for the accelerator, confirming that the remaining operations (attention, normalization, and residual additions) set a hard throughput ceiling near 8 tokens per second on this platform.

Based on this analysis, the proposed system focuses on offloading the most computationally intensive component of the model to an FPGA-based accelerator while leaving control logic and lightweight operations on the CPU. This selective acceleration strategy enables significant reduction of inference latency at low thread counts without requiring full hardware implementation of the entire model.

2.3 Orchestration of the Computation Handoff

Following the identification of the dominant computational bottleneck, the llama.cpp inference engine was extended with a hardware offloading mechanism that intercepts the execution of the feed-forward MatMul + SwiGLU block and orchestrates a handoff to a custom accelerator implemented in the FPGA fabric. Inference is executed using a modified `llama.cpp` CPU backend. Model weights are loaded from GGUF, and for each token a GGML computation graph is built (attention, norms, FFN). The graph is executed node-by-node via `ggml_graph_compute`, which dispatches each operation to CPU kernels by default.

To accelerate the dominant FFN block, the LFM2 builder replaces the standard FFN sequence with a fused operator (`ggml_swiglu_fused_hw`) when the environment variable `LLAMA_SWIHW=1` and the tensor shapes and types match the accelerator interface. The layer index is stored in the node to retrieve the correct weights. At execution time, this fused node triggers a custom kernel that manages CPU–FPGA interaction. Weights are cached once per layer in a physically contiguous, DMA-coherent buffer (udmabuf) accessible by both the CPU and the FPGA, and are never re-transferred after the first use. The current activation vector is quantized from FP32 to INT8 on the CPU. The CPU programs the accelerator via memory-mapped registers, asserts a start signal, and sleeps until an interrupt from the FPGA fabric signals completion. The FPGA computes the full MatMul + SwiGLU block in a pipelined dataflow and writes the FP32 result back to the DMA buffer, from which the CPU copies it into the GGML output tensor. All other layer operations remain on the CPU. If conditions are not met at graph-build time or `LLAMA_SWIHW` is set to 0, execution falls back to the standard CPU path.

Algorithm 1 LFM2.5 Inference Execution

load_model(GGUF)
for each token **do**
  $graph \leftarrow$ build_ggml_graph(token)
  for each node in $graph$ **do**
    if node.op == SWIGLU_FUSED_HW and enabled **then**
      // CPU–FPGA handoff
      if not $weights\_cached[layer]$ **then**
        Copy $W_{gate}, W_{up}, W_{down} \rightarrow shared\_mem$
        Mark cached
      end if
      $x_{int8}, x_{scale} =$ quantize(x)
      Write $x_{int8} \rightarrow shared\_mem$
      program_fpga($W_{addrs}, x_{addr}, out_{addr}, mode, x_{scale}$) {write AXI-Lite ctrl regs}
      start_fpga()
      wait_for_interrupt()
      $out =$ read_output($shared\_mem$)
    else
      $out =$ cpu_kernel(node)
    end if
  end for
  {graph execution complete; proceed to next token}
end for

Although Algorithm 1 expresses the offload as a single conditional inside the per-node loop, its effect over a full token is dictated by the block structure of Section 2.1.2. The computation graph of one token contains sixteen SwiGLU FFN nodes, one per transformer block, and the driver intercepts every one of them. The offload is therefore not a single event but a sequence of sixteen separate, synchronous CPU–FPGA handoffs per generated token.

These handoffs cannot be fused or chained on the FPGA, because consecutive FFNs are never adjacent in the graph. The output of the FFN in block N is consumed by the residual addition, the RMSNorm, and the sequence mixer of block $N{+}1$, all of which execute on the CPU; only their result becomes the input activation of the FFN in block $N{+}1$. The accelerator thus has no way

to advance from one FFN to the next on its own: the CPU lies on the critical path between every pair of feed-forward blocks. As a consequence the two devices strictly alternate: while the FPGA evaluates one FFN the CPU is idle waiting on the completion interrupt, and while the CPU computes a block's mixer and normalizations the FPGA is idle, so neither device overlaps the other within a token. Figure 2.2 details the protocol of a single handoff.

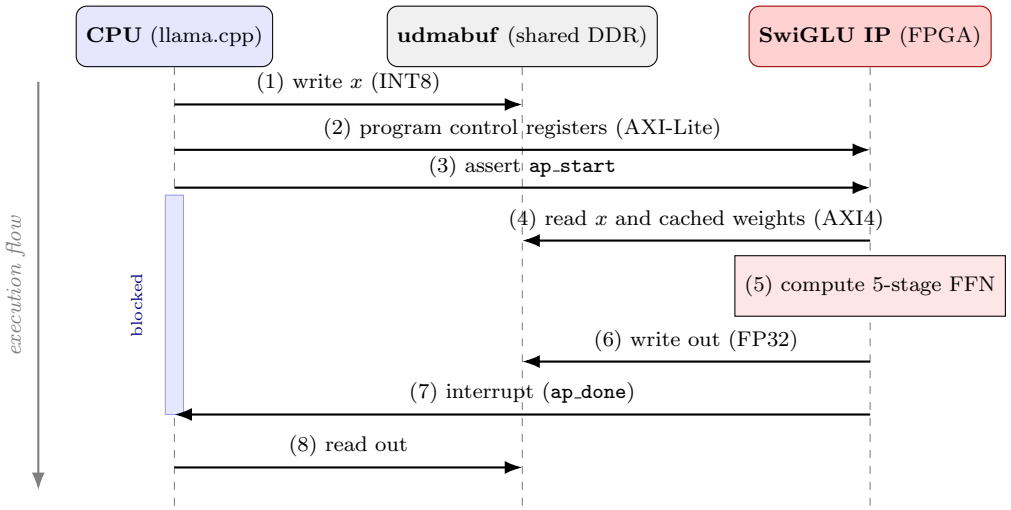

Figure 2.2: Protocol of a single CPU–FPGA handoff. The CPU quantizes the activation and writes it to the shared udmabuf buffer, programs the IP's AXI-Lite control registers, asserts `ap_start` and blocks; the FPGA burst-reads the activation and the cached weights, evaluates the five-stage SwiGLU pipeline, writes the result back, and raises a completion interrupt that releases the CPU to copy the output. This sequence repeats sixteen times per generated token.

Each of the sixteen handoffs carries a fixed overhead that is independent of the work it guards: the input activation is quantized to INT8, the AXI-Lite control registers are programmed, the shared buffer is synchronized for the device, the calling thread blocks until the completion interrupt, and the result is copied back into the GGML tensor. This cost is paid sixteen times per token and does not amortize with sequence length the way the arithmetic of a batched matrix multiply would, since decode generates one token at a time. It is the principal component of the ≈ 126 ms of non-FFN per-token overhead measured in Section 2.2, which fixes the Amdahl ceiling near 8 tokens per second. The synchronous, blocking nature of the handoff also explains the behaviour at high thread counts (Section 3.5): when all four cores are busy, the blocking wait competes with the interrupt handler for scheduler time and the per-token round-trip latency grows, so the FPGA path can fall below the CPU-only baseline. Overlapping FPGA execution with independent CPU work would require an asynchronous, pipelined handoff in place of the node-by-node, barrier-synchronized execution used here; this direction, and the compatibility trade-off it entails, is discussed in Chapter 4.

2.4 FPGA Accelerator

The proposed accelerator implements the SwiGLU feed-forward block of the LFM2.5 language backbone,

$$\mathrm{FFN}(x) = \left(\mathrm{SiLU}(xW^T) \odot (xV^T)\right) W_{down}^T, \tag{2.2}$$

which the five-stage dataflow pipeline evaluates through the decomposition:

$$X_1 = xW^T \tag{2.3a}$$
$$X_2 = xV^T \tag{2.3b}$$
$$G = \mathrm{SiLU}(X_1) \odot X_2 \tag{2.3c}$$
$$\mathrm{FFN}(x) = GW_{down}^T \tag{2.3d}$$

where x is the input activation, W is the gating projection, V is the expansion projection, and W_{down} is the output projection; $\odot$ is element-wise multiplication and $\text{SiLU}(z) = z \cdot \sigma(z)$. Equation (2.3a) is computed in Stage 2a (gate projection), Eq. (2.3b) in Stage 2b (up projection), Eq. (2.3c) in Stages 3–4 (gated activation: SiLU sub-stage A, element-wise multiply sub-stage B), and Eq. (2.3d) in Stage 5 (down projection).

The design exposes AXI4 interfaces for weights and activations, and an AXI-Lite interface for runtime control (quantization mode and per-token scaling factors). The five-stage pipeline is organized as an HLS dataflow (Fig. 2.3) targeting an initiation interval $II = 1$ throughout; end-to-end latency is therefore determined by memory bandwidth and burst efficiency rather than arithmetic capacity.

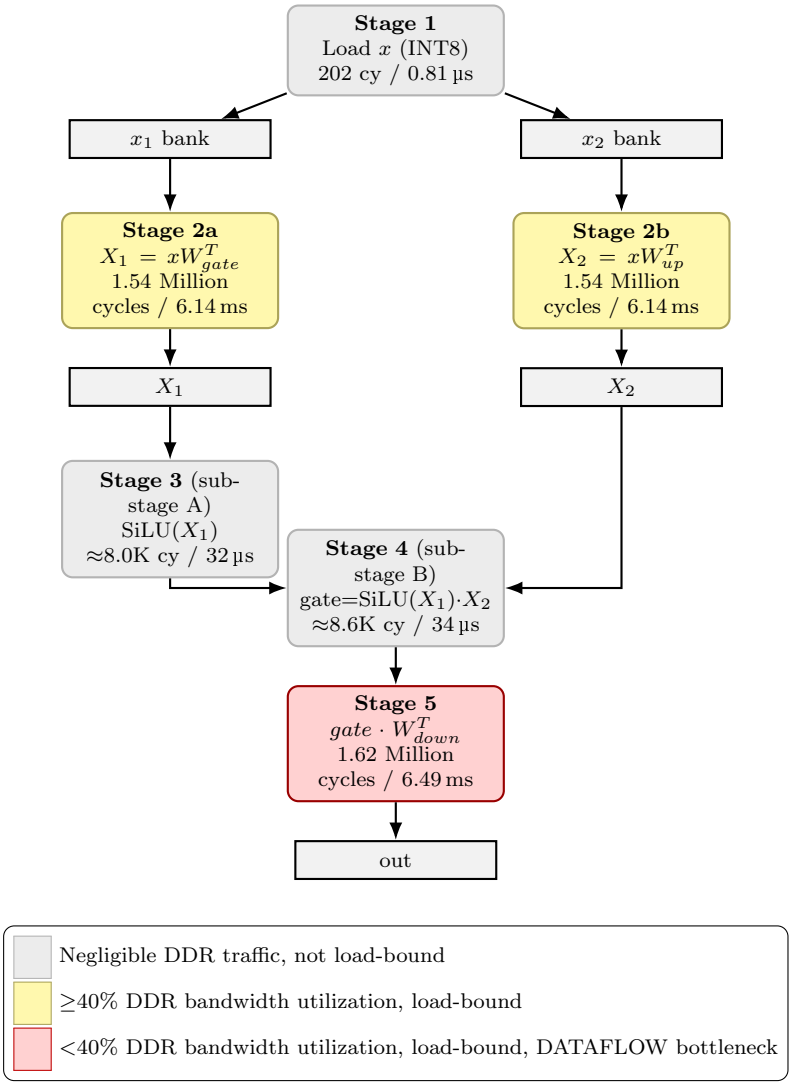

Figure 2.3: Compact dataflow of the MatMul + SwiGLU accelerator.

2.4.1 HLS Realization

The accelerator is written in C++ and compiled to register-transfer-level hardware by Vitis HLS. Its functional behaviour is expressed in ordinary loops and arithmetic, while the mapping to hardware (interface protocols, on-chip memories, parallelism, and pipelining) is steered by directives, or pragmas, that annotate the code without altering its result. Listing 2.1 shows the top-level function, which both defines the IP's external interface and launches the five pipeline stages.

Four kinds of directive appear. The `m_axi` pragmas turn the weight and activation pointers into AXI4 master ports. Because HLS otherwise infers the bus width from the `uint8_t` element type, giving an 8-bit bus, the `max_widen_bitwidth=128` option is what widens each port to 128 bits and raises the achievable DDR bandwidth roughly sixteen-fold; this is the bandwidth on which the load-bound analysis of this chapter rests. The `s_axilite` pragmas expose the scalar arguments (the weight and

```
void swiglu(const uint8_t *W, const uint8_t *V, const uint8_t *W_down,
            const int8_t *x_batch, float *out_batch,
            uint32_t down_quant_mode, float x_scale) {
  // weight/activation pointers become 128-bit AXI4 master ports to DDR
  #pragma HLS INTERFACE m_axi port=W bundle=gmem_W max_widen_bitwidth=128
  // ... V, W_down, x_batch, out_batch likewise ...
  // scalar arguments become memory-mapped control registers
  #pragma HLS INTERFACE s_axilite port=x_scale bundle=CTRL
  // ... addresses, down_quant_mode, ap_ctrl likewise ...

  int8_t X1_cache[FFN_DIM];
  #pragma HLS BIND_STORAGE    variable=X1_cache   type=ram_2p impl=bram
  int8_t gate_cache[DOWN_BLOCKS][256];
  #pragma HLS BIND_STORAGE    variable=gate_cache type=ram_1p impl=uram
  #pragma HLS ARRAY_PARTITION variable=gate_cache dim=2 cyclic factor=8

  #pragma HLS DATAFLOW                          // stages run as overlapping processes
  load_x_local(x_batch, x_local_1, x_local_2);
  compute_X1(W, x_local_1, x_scale, X1_cache);   // Stage 2a
  compute_X2(V, x_local_2, x_scale, X2_cache);   // Stage 2b
  compute_gate(X1_cache, X2_cache, gate_cache);  // Stages 3-4
  compute_output(W_down, gate_cache, out_batch); // Stage 5
}
```

Listing 2.1: Abridged top-level HLS function: the external interface and the five-stage dataflow. Repeated `m_axi` and `s_axilite` lines are collapsed for clarity.

buffer addresses, the quantization mode, the per-token scale, and the `ap_ctrl` start/done handshake) as the memory-mapped control registers that the driver programs during the handoff of Section 2.3. The `BIND_STORAGE` directives pin each on-chip array to a specific primitive, LUTRAM, BRAM, or URAM, so that the design fits the ZU5EV's mixed memory budget; `X1_cache` is a dual-port (`ram_2p`) BRAM because it is written by `compute_X1` and read by `compute_gate` concurrently. `ARRAY_PARTITION` with `cyclic factor=8` splits the gate cache into eight URAM banks so that eight blocks can be read in a single cycle, feeding the parallel multiply-accumulate chains. Finally, `DATAFLOW` instructs the tool to run the five stage functions as concurrent, overlapping processes rather than in sequence: this is what realizes the pipeline of Figure 2.3, with the two projection stages executing in parallel.

The remaining directives operate inside the stage functions: each accumulation loop is pipelined at an initiation interval of one (`PIPELINE II=1`) with the block loop unrolled (`UNROLL`), so the partitioned weight and activation banks are consumed in parallel. This is the mechanism behind the 32 multiply-accumulate chains detailed in the following subsections.

2.4.2 Quantization

Q4_K is preferred over the structurally simpler Q4_0 format (one fp16 block scale per 32 elements, no sub-block scales) on quality grounds: its 6-bit sub-group scales provide 16× finer scale resolution per block, keeping quantization fidelity above the Q4_0 baseline that Liquid AI themselves use for all LFM2 inference benchmarks [4]. The default GGUF distribution of LFM2.5-1.2B applies mixed quantization to the down-projection weights, using Q6_K for eight layers and Q4_K for the remaining eight (the Q4_K_M strategy). Supporting Q6_K in the accelerator is infeasible on the ZU5EV: each Q6_K block occupies 210 bytes (13.125×128-bit words), precluding the clean 2D register-file layout that enables compile-time `.range()` nibble extraction. The fallback `get_byte()` multiplexer tree required for 6-bit nibbles consumes approximately 30–40 K LUTs per MAC stage at 32-block parallelism, exceeding the device's remaining routing capacity at 87% LUT occupancy. The model is therefore re-quantized to uniform Q4_K across all sixteen layers using `llama-quantize --allow-requantize --pure` from an F16_Q8_0 source. This choice is consistent with the hardware-in-the-loop co-design process of LFM2, whose architecture search and published inference benchmarks evaluated all candidates under Q4_0 quantization [4], a scheme coarser than Q4_K in both bit-width and block-scale granularity. Furthermore, Q6_K is applied exclusively to the down-projection W_{down}, the least quantization-sensitive

weight in the FFN: it acts as a linear readout of an already-nonlinear gated intermediate, and its 32-block structure averages quantization error before injection into the residual stream. Across the full mixed-precision datapath, accuracy degradation relative to the CPU's FP32 reference is estimated from quantization error bounds: the dominant contribution is Q4_K weight error (approximately 6% relative to the FP32 dynamic range), with INT8 input quantization adding under 0.5% and INT8 intermediate cache quantization contributing at most 1%. These are order-of-magnitude bounds, not direct measurements on this model; the impact on task-level accuracy is expected to be within the tolerance of a conversational audio deployment.

2.4.3 CPU-Side Weight Pre-Decoding

The $K = 4$ designation indicates that four weight rows are processed per iteration of each MAC stage, yielding 32 parallel INT32 multiply-accumulate chains (8 blocks $\times$ 4 rows) in the projection stages and 32 chains shared across 4 sequential groups in the down-projection stage. Sustaining this degree of parallelism within the device's LUT budget is made possible by a pre-decoding step that runs on the CPU, once per layer, before the weights are ever seen by the accelerator.

The difficulty stems from the Q4_K storage format, which is designed for CPU SIMD execution rather than for an FPGA datapath. Each 144-byte block encodes 256 weights as shown in Table 2.3: two fp16 block-level scales (d and $dmin$), eight 6-bit sub-block scales and eight 6-bit minima (`sc6`, `mn6`) packed at bit-level offsets that straddle byte boundaries, and 256 four-bit nibbles stored in block-major order. Decoding this layout inside the MAC pipeline would require, for every nibble, a `get_byte()` access, a nine-to-one multiplexer on the 128-bit word followed by a four-bit shift. With eight blocks decoded in parallel, these multiplexer trees alone consume on the order of 30 K LUTs, which is incompatible with the $K = 4$ design on the ZU5EV.

Table 2.3: Layout of a 144-byte Q4_K block (256 weights).

Bytes	**Size**	**Field**
0–1	2 B	d (fp16 block scale)
2–3	2 B	$dmin$ (fp16 min-subtraction scale)
4–11	8 B	`sc6`/`mn6` sub-block scales and minima (low 6 bits)
12–15	4 B	`sc6`/`mn6` high-order bits (interleaved)
16–143	128 B	256 packed 4-bit nibbles (block-major)

The pre-decoding step performs two transformations that move all of this irregular bit manipulation off the critical path and onto the CPU. First, the interleaved 6-bit `sc6` and `mn6` fields are unpacked into flat INT8 arrays, so the FPGA reads each sub-block scale as a plain byte. Second, and more importantly, the nibbles are transposed from the block-major order in which Q4_K stores them into an element-major order, illustrated in Figure 2.4: instead of all 256 nibbles of block 0 followed by all 256 of block 1, the pre-decoded layout places the nibble of a given element for all eight blocks side by side in a single 128-bit word. The fp16 scales d and $dmin$ are copied verbatim and converted to fp32 in hardware by a custom bit-manipulation routine that preserves subnormal values rather than flushing them to zero.

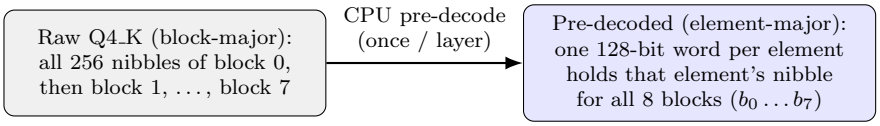

Figure 2.4: Weight pre-decoding rearranges the block-major Q4_K nibbles into an element-major layout, so each 128-bit memory word holds one element's nibble for all eight blocks and can be consumed by the parallel MAC chains in a single cycle.

After this rearrangement the FPGA load is trivial: each 128-bit DDR word is written verbatim into a wide (128-bit) BRAM tile, one read and one write per cycle, achieving an initiation interval of one with no decode logic at all. In the MAC, the eight nibbles of a word are addressed by compile-time `.range()` slices, which synthesize to wires and cost zero LUTs. Eliminating the multiplexer trees is

what brings the $K = 4$ implementation from roughly 137 K LUTs (which does not fit the ZU5EV's 117 K) down to roughly 103 K, leaving margin for routing; the implemented design occupies 87% of the LUTs (Section 3.3). The pre-decode itself costs about 10 ms per layer on the first token; its output is cached permanently in the shared udmabuf buffer, so every subsequent token reads the transposed weights directly and pays nothing.

2.4.4 Stage 1: Load x (INT8)

The 2048-element INT8 input vector is burst-loaded from DDR in 128×128-bit AXI words and duplicated into two identically partitioned LUTRAM buffers, enabling concurrent access by Stages 2a and 2b without serialization. The stage is pure data movement with no arithmetic and completes in 202 cycles (0.81 μs).

2.4.5 Stages 2a / 2b: Linear Projections (Parallel)

Two GEMV operations compute X_1 and X_2 (Eqs. (2.3a)–(2.3b)) in parallel on independent AXI ports and BRAM banks. Weights are pre-decoded on the CPU into flat INT8 sub-block scales and element-major nibble words; the FPGA never handles the interleaved Q4_K header format. With $K = 4$, four rows are processed per outer iteration via a single merged 320-word AXI burst covering all four rows simultaneously (16 header + 64 nibble words per row, $\times 4$ rows = 320 total), eliminating the per-iteration burst-setup overheads of a design with multiple calls. Nibble extraction uses compile-time `.range()` indexing on wide 128-bit BRAM tiles. The 32 parallel INT32 MAC chains (8 blocks × 4 rows) deliver 2.76 GOPS (34.5% of the 8.00 GOPS peak); the merged burst reduces load time from 948 to 401 cycles per iteration, representing 53% of the total per-iteration cycle budget. DDR bandwidth reaches 1.63 GB/s (41% of the 4.00 GB/s peak) across the 10.0 MB per-matrix transfer. Each stage completes in 1,536,001 cycles (6.14 ms).

2.4.6 Stages 3–4: Gated Activation and Quantization

The fused gate operation (Eq. (2.3c)) runs entirely on-chip from the X_1 and X_2 caches using a two-pass scheme over 8,192 elements: the first pass finds the maximum absolute value across 8 parallel reduction lanes, and the second recomputes and requantizes to INT8 for storage in the 8-way banked URAM gate cache. At 16,622 cycles (66 μs) and approximately 65K FP operations per pass, the stage contributes under 1% of per-layer time.

2.4.7 Stage 5: Down Projection

The gated activation is projected back to model dimension (Eq. (2.3d)) via out = gate$\cdot W_{down}^T$ using the same CPU-pre-decoded weight format as Stages 2a/2b. With $K = 4$, four output rows are processed per iteration across 32 blocks organized into 4 sequential groups of 8; all four rows are unrolled per group to deliver 32 parallel INT32 MAC chains per group. Unlike Stages 2a/2b, which merge all four rows into a single 320-word burst, Stage 5 issues four sequential per-row AXI bursts of 320 words each. The merged strategy is not applicable here: W_{down} has 32 blocks per row (versus 8 in W/V), so each row's nibble buffer is 256 words rather than 64. Consequently, nibbles are stored in narrower 32-bit tiles (4 groups × 4 rows) rather than 128-bit tiles. This tile layout also precludes the contiguous multi-row memory mapping that enables burst merging in Stages 2a/2b. These loads account for 60% of the per-iteration budget (1,908 of 3,165 cycles), making the stage load-bound. DDR bandwidth reaches 1.54 GB/s (38% of the 4.00 GB/s peak) across the 10.0 MB transfer. Total latency is 1,622,602 cycles (6.49 ms), which sets the dataflow pipeline interval.

3 Experiments and Discussion

3.1 Experimental Setup

Hardware Platform. Experiments were conducted on the Processing System (PS) of the Kria KV260, an embedded UMA platform featuring a quad-core CPU running at 1.3 GHz and an FPGA fabric operating at 250 MHz. The system is equipped with 4 GB of shared memory. Power measurements were collected using onboard sensors sampling at runtime via the AMD `xmutil` toolchain.

Software Stack. The inference engine is based on a modified version of `llama.cpp`, extended to support hardware offloading of the MatMul + SwiGLU block. Models are executed using the Q4_K quantization format. The FPGA accelerator was synthesized using the Vitis high-level synthesis (HLS) toolchain and integrated in the Vivado Design Suite.

Workload. The evaluation uses the LFM2.5-1.2B language backbone (input dimension 2048, feed-forward dimension 8192), which is shared across the Instruct and Thinking fine-tuned variants and the base version. The accelerator processes one token per invocation, with multi-token prefill handled by sequential IP calls in the driver loop, issued by the modified `llama.cpp`. This reflects the single-token synthesis constraint imposed by on-chip memory limits.

Baseline. The baseline corresponds to CPU Only execution of the same model through an identical inference pipeline. In the accelerated configuration, only the MatMul + SwiGLU block is offloaded to the FPGA, while all remaining operations are executed on the CPU. Both configurations are produced by the same modified `llama.cpp` binary, with the offload enabled or disabled at runtime through the `LLAMA_SWIHW` environment variable (`1` for FPGA+CPU, `0` for CPU Only), without recompilation. The CPU Only baseline uses the standard Q4_K_M distribution, whereas the FPGA+CPU path uses the uniformly re-quantized Q4_K model that the accelerator requires (Table 3.1).

3.2 Evaluation Metrics

The evaluation captures both performance and efficiency through a set of directly measured and derived quantities. Throughput, expressed in tokens per second, is measured separately for the prefill and decode phases using `llama-bench` with a fixed workload of 12 prompt tokens and 64 generated tokens, repeated over 10 runs to obtain stable numbers. Power consumption is recorded concurrently via the onboard telemetry interface (`xmutil platformstats`, an integrated system monitoring utility on the KV260 platform), which samples the system-on-module total power at 1 Hz throughout each benchmark session. From these two measurements, performance per watt (tokens/s/W) is derived as the ratio of throughput to mean power, providing a joint view of speed and energy efficiency across operating points. FPGA resource utilization, reported in terms of look-up tables (LUT), flip-flops (FF), block RAM (BRAM), ultra RAM (URAM), and DSP slices, is taken from the post-place-and-route reports produced by the Vivado implementation flow and quantifies how closely the accelerator design approaches the physical limits of the target device. Together, these metrics characterize the trade-off space between throughput, efficiency, and resource utilization that governs the feasibility of edge deployment.

3.3 Results

This section compares CPU Only and FPGA+CPU execution across thread counts (T1–T4) using throughput and performance-per-watt. Table 3.1 reports the main quantitative results, while Fig. 3.1 provides hardware resource usage.

Peak resident memory is approximately 728 MB for CPU execution (the GGUF model is memory-mapped and demand-paged; measured peak Resident Set Size 728 MB across all thread counts) and approximately 1.4 GB for the FPGA+CPU path (the same 728 MB process RSS plus a 640 MB CMA-

Table 3.1: Throughput and efficiency across platforms and thread counts. Bold marks the preferable value per metric within each KV260 thread-count pair.

Threads	Mode	Prefill (t/s)	Decode (t/s)	Power (W)	Prefill (t/s/W)	Decode (t/s/W)
KV260 (4× Cortex-A53, CPU Only: Q4_K_M | FPGA+CPU: Q4_K, 250 MHz FPGA fabric)						
T1	CPU Only	1.09	0.88	3.96	0.275	0.222
T1	FPGA+CPU	**2.20**	**1.63**	5.10	**0.431**	**0.320**
T2	CPU Only	2.17	1.66	4.25	0.511	0.391
T2	FPGA+CPU	**2.90**	**2.21**	5.15	**0.563**	**0.429**
T3	CPU Only	**3.16**	**2.36**	**4.41**	**0.717**	**0.535**
T3	FPGA+CPU	3.06	2.05	5.14	0.595	0.399
T4	CPU Only	**3.52**	**2.31**	**4.28**	**0.822**	**0.540**
T4	FPGA+CPU	2.85	1.46	5.13	0.556	0.285
STM32MP257F-DK (2× Cortex-A35, Q4_K_M, power analytically estimated[b])						
T1	CPU Only	0.83	0.67	0.62	1.339	1.081
T2	CPU Only	1.61	1.28	0.80	2.013	1.600

[b]Derived from STM32MP257 datasheet characterization [31] (Tables 22–26) and PMIC rail voltages [32] (§7.5.3); see Section 3.4 for full derivation.

allocated udmabuf DMA buffer that does not appear in RSS). Per-token decode latency (the reciprocal of decode throughput) ranges from 1136 ms at T1 CPU Only down to 424 ms at T3 and 433 ms at T4; for FPGA+CPU it is 613 ms at T1, 452 ms at T2, 488 ms at T3, and 685 ms at T4.

Figure 3.1: Utilization of the Kria KV260 on-board resources for the implementation of the feed-forward block accelerator.

3.4 Comparison with STM32MP257F-DK

To contextualize the KV260 results against a different Arm-class edge platform, the standard LFM2.5-1.2B Q4_K_M distribution (8 of 16 down-projection layers at Q6_K, the remainder at Q4_K) was benchmarked on the STM32MP257F-DK, which integrates on chip two Cortex-A35 cores running at up to 1.5 GHz. Results are reported alongside the KV260 data in Table 3.1. Total board power consumption is derived analytically as the sum of four terms:

(i) **SoC rail power**: datasheet characterization currents [31] (Tables 22–26, $T_j = 25\,^{\circ}\mathrm{C}$) multiplied by the PMIC output voltages programmed in the board user manual [32] (§7.5.3, $V_{\mathrm{DDCPU}} = 0.91\,\mathrm{V}$, $V_{\mathrm{DDCORE}} = 0.82\,\mathrm{V}$); this yields 174 mW at T1 and 320 mW at T2,

(ii) **LPDDR4 power**: background-dominated at fewer than 5% utilization ($\approx 150\,\mathrm{mW}$ independent of thread count),

(iii) **Board peripheral idle power** (GigE PHY, STLINK, Wi-Fi module, eMMC): $\approx 205\,\mathrm{mW}$, constant across T1 and T2,

(iv) **PMIC buck conversion losses** at the operating load, assuming $\eta = 0.86$ for the SoC/DRAM rails and $\eta = 0.88$ for the 3.3 V peripheral rail.

Summing all terms gives a total board input power of 0.62 W (T1) and 0.80 W (T2).

3.5 Discussion

The FPGA+CPU solution improves efficiency at lower thread counts but not uniformly. At T1, performance-per-watt improves by +57% (prefill) and +44% (decode) versus CPU Only, with board power rising from 3.96 W (CPU) to 5.10 W (FPGA+CPU). At T2, the gains narrow to +10% for both phases (4.25 W versus 5.15 W). At T3 and T4, CPU Only is superior: FPGA+CPU is lower by 17%/25% at T3 and 32%/47% at T4 (prefill/decode). At T4, FPGA+CPU decode degrades further to 1.46 t/s (below the T3 value of 2.05 t/s), because all four A53 threads compete with the UIO interrupt handler for scheduler time, increasing per-token round-trip latency. These trends are consistent with end-to-end throughput being constrained more by orchestration and memory movement than by arithmetic scaling alone.

The results indicate that offloading the MatMul + SwiGLU feed-forward block to the FPGA is beneficial because it targets a runtime-dominant kernel, consistently with prior transformer accelerators that focus on FFN/MLP and attention substructures as the most advantageous hardware targets [26]. The accelerator is effective when host-side contention is limited and the cost of orchestration can be amortized (T1–T2); at higher thread counts (T3–T4), the incremental benefit is offset by transfer overhead and synchronization, and the CPU alone can supply and process the residual stream more efficiently.

Comparing the KV260 against the STM32MP257F-DK isolates the effect of the wider Cortex-A53 pipeline and FPGA fabric from the Cortex-A35 baseline. The A35 achieves 1.28 t/s decode at T2, versus 1.66 t/s for the KV260 A53 at T2 (CPU Only) and **2.21** t/s for the KV260 FPGA+CPU path. This places the KV260 FPGA+CPU configuration at a **1.7**× decode speedup over the A35 at matched thread count, at the cost of a more complex integration and approximately **4.4** W of additional board power (5.15 W FPGA+CPU versus 0.80 W for the STM32 at T2). The STM32's 3.7× efficiency advantage (1.60 t/s/W decode versus 0.429 t/s/W for the KV260 FPGA+CPU) confirms that the accelerator is justified by its latency margin: the 1.7× decode speedup brings the system within the 452 ms real-time window that the A35 at 781 ms per token cannot reach.

Section 1.2 defined a per-token generation latency target of 200–500 ms for the language model component. Of the eight KV260 configurations, four enter this range: T3 and T4 CPU Only (424 and 433 ms) and T2 and T3 FPGA+CPU (452 and 488 ms). No configuration reaches the 200 ms instantaneous threshold. The FPGA+CPU path achieves sub-500 ms decode with only two active CPU threads, whereas CPU Only requires three or four to enter the same latency range. This thread advantage matters in practice: a full conversational pipeline must concurrently run ASR and TTS stages that compete for the same A53 cores, so reducing the language model's thread demand directly widens the scheduling budget for the surrounding stages. At the same time, the 452 ms LM-only latency at T2 FPGA+CPU is already a significant fraction of the 500 ms budget, leaving little headroom for ASR and TTS; meeting the 0.5 s aggregate target in a streaming pipeline will require either a faster platform or accepting the "immediate" perceptual range (0.5–1 s) defined in Section 1.2.

The contrast highlights that the practical value of the FPGA offload scales with the gap between the host CPU and the accelerator: on a weaker host the absolute gain is larger, while on a stronger host (more A53 threads) the orchestration overhead increasingly dominates. Table 3.2 places this

design within the space of published KV260 accelerators; unlike compute-intensive matrix-multiply designs that saturate DSP slices, the MatMul + SwiGLU kernel saturates routing fabric (87% LUT, 20% DSP), reflecting a load-dominated rather than arithmetic-dominated workload.

Table 3.2: KV260 accelerator design comparison.

	Tiny Transf. [10]	**Self-Attn.** [25]	**MatMul Accel.** [33]	**This Work**
Accelerated function	Full E2E pipeline	Q/K/V projections	Matrix mult.	MatMul + SwiGLU
Model / matrix size	6.6k params.	DistilBERT	DistilBERT	LFM2.5 1.2B
Model size (params)	6.6k	66M	66M	1.2B
Precision	float32	INT8	INT8	INT8 / INT4
Clock (MHz)	77	100	100	250
Ops/cycle (INT8 MAC)	—	1024	1024	32
Pure compute throughput (GOPS)[d]	—	3.12[c]	22.83[c]	8.00[a]
Effective system throughput (GOPS)[e]	—	2.85	13.52	2.58[b]
LUT (%)	30	60	84	87
DSP (%)	27	83	83	20
BRAM (%)	37	88	71	32
FF (%)	20	43	44	50
URAM (%)	—	—	—	13
Power (W)	—	3.3	—	4.8–6.0

[a]32 parallel INT32 MAC chains (8 blocks × 4 rows) at 250 MHz. [b]Stage 5 (down projection). [c]Evaluation uses an FFN matrix multiplication case with dimensions $(64, 768) \times (768, 3072)$. [d]Hardware-only peak; excludes memory transfer and host CPU overhead. [e]Includes all system-level effects: memory transfer, host CPU communication, and orchestration.

Results reported using larger FPGA platforms suggest that greater memory bandwidth and fabric capacity help kernel-level gains and end-to-end speedup [23]. Direct comparison with the 6.6k-parameter Tiny Transformer on the same platform highlights the resource intensity of conversational architectures. While the Tiny Transformer maps its entire end-to-end pipeline at low resource utilization across all fabric categories (Table 3.2), accelerating just the feed-forward MatMul + SwiGLU block of the 1.2B-parameter LFM2.5 model nearly saturates the same device at 87% LUT and 32% BRAM utilization at 250 MHz. Critically, the Tiny Transformer's small parameter count is not merely a resource advantage [10]: by fitting the entire pipeline on chip, it eliminates off-chip weight fetching entirely, allowing execution to remain compute-bound and sustain near-peak arithmetic throughput, a regime that larger models cannot achieve on the same device. The Self-Attention accelerator [25] highlights differing architectural trade-offs: its two-level tiling strategy keeps both matrix operands within on-chip BRAM across the Q, K, and V projections, allowing a fully-unrolled 32×32 MAC array to sustain 1024 concurrent INT8 operations per cycle without off-chip bandwidth pressure, reaching 91% effective utilization of peak throughput. This is achievable because the DistilBERT-scale projection matrices fit within the on-chip tiling footprint. The LFM2.5 SwiGLU weights, by contrast, total approximately 30 MB per layer across the three Q4_K projections, far beyond the ZU5EV's combined on-chip capacity of roughly 3.4 MB (block, ultra, and distributed RAM combined), which makes DDR streaming unavoidable and places the design in a memory-bandwidth-limited regime regardless of MAC array width or tiling strategy. A comparable BRAM-banked matrix multiplication study [33] reports similar severe PS-side bottlenecks, reaching only 59% of its peak throughput; the present work reaches 34–41% of peak capacity, reflecting its higher per-token weight volume. The contrast in DSP usage (83% versus 20%) reflects an operand-width mismatch rather than a deliberate design choice: the MatMul + SwiGLU kernel's core MAC operation multiplies a 4-bit nibble weight by an INT8 activation, yielding a 12-bit product too narrow to use a DSP48E2 slice efficiently. The DSP48E2 is optimized for 18×27-bit multiplications, so HLS maps the narrow multiply-accumulate operations to LUT-based logic instead; the 20% DSP usage in this work arises entirely from the FP32 scale and minimum-factor accumulation paths, not from the dominant weight–activation inner loop.

Extending hardware acceleration beyond the MatMul + SwiGLU block to the Final Linear Projection and Gated Short Convolution yields diminishing returns, as they account for only 10.96% and 10.44% of total execution time, respectively. Furthermore, simultaneous acceleration is physically infeasible on the KV260, as the MatMul + SwiGLU kernel alone leaves insufficient resources for

additional hardware mapping. To characterize the attainable design space, roughly a dozen distinct architectural variants were synthesized and evaluated during development, systematically varying the nibble storage width, the BRAM banking strategy, the AXI burst organization, and the use of function inlining and stream decoupling. Table 3.3 summarizes the most informative attempts. Their consistency is itself the main finding: three independent ceilings emerged, and every variant that attacked one of them converged on the same wall.

Table 3.3: Representative architectural variants explored during development and the ceiling each one revealed.

Strategy	**Result**	**Ceiling revealed**
Wide-BRAM nibble storage (gate/up)	$II = 1$ load, 67 cycles/row; enables $K = 4$ within budget	final choice
URAM → BRAM swap	No change	Four-write limit is operation-count, not latency
Cyclic-4 banking (proves independence)	No change	Four-write limit, not bank aliasing
Stream-decoupled dataflow	No change	Four-write fanout, not the AXI engine
Merged 4-row burst (gate/up)	−38% cycles but +14K LUT → 138K (118%)	AXI/LUT budget: place-and-route fails
Wide-BRAM output load	−15% cycles, 74K routing overlaps	ZU5EV routing capacity
Full output-MAC unroll	+102% LUT, negligible speedup	ZU5EV routing/LUT capacity

The first ceiling is AXI burst serialization. Each DDR burst pays a fixed setup overhead of approximately 45 cycles on the ZU5EV's HP-port interconnect before data begins to flow, so a single contiguous burst at an initiation interval above one outperforms several parallel bursts at $II = 1$. A merged load that fetches all four rows of Stages 2a/2b in one 320-word burst does reduce the projection-stage cycle count by 38%, but the four-way header-routing logic it requires adds about 14 K LUTs, pushing the design to 138 K (118% of the device) and failing place-and-route. The single-burst organization is therefore retained, and the down-projection, whose rows are four times longer, cannot apply the merge at all.

The second ceiling is the HLS scheduler's refusal to issue four memory writes in one pipeline stage. Because the down-projection stores its nibbles in narrow 32-bit tiles rather than the wide-BRAM layout used elsewhere (Section 2.4.3), each 128-bit DDR word must be split and written into four separate single-port buffers within the same cycle. Vitis HLS will not schedule four writes per cycle, and, crucially, this is a limit on the *number* of memory operations, not a physical port conflict: a cyclic partitioning that proves the four banks are mutually independent does not help, and swapping the buffers between BRAM and URAM changes nothing. The gate and up projections escape the limit by storing each pre-decoded 128-bit word verbatim, a single write per cycle; the down-projection cannot, because its 32 blocks per row (the 8192-element contraction dimension divided by the 256-weight Q4_K block) demand 32 simultaneous block, header, and scale reads in the MAC, a density the ZU5EV fabric cannot route in the wide-BRAM form.

That routing density is the third ceiling. A fully-unrolled output MAC array, fed by wide-BRAM reads, gate-cache lookups, and sub-block-scale accesses, exceeds the device's routable interconnect at 250 MHz: the wide-BRAM output variant reaches its target cycle count in synthesis but leaves more than 74,000 unresolved routing-node overlaps in implementation. A migration to the ZU7EV (the next device in the same Zynq UltraScale+ family, with approximately 230 K LUTs on the same processor–fabric interface) would provide the headroom to route the fully-unrolled array and to merge the load calls that currently serialize the AXI path.

Taken together, these ceilings bound the achievable throughput at approximately 3.6 t/s, well within the Amdahl ceiling of ∼8 t/s established in Section 2.2; the deployed design attains 61% of this 3.6 t/s design ceiling. They reflect the structural mismatch between a memory-bandwidth-limited workload and a device optimized for logic-dense, compute-bound kernels.

Table 3.4 consolidates the latency compliance, efficiency outcome, and dominant limiting factor for each operating point discussed above, while Table 3.5 summarizes the architectural constraints that bound the throughput ceiling independent of thread count.

Table 3.4: Operating-point summary: per-token decode latency, real-time budget compliance, efficiency outcome, and dominant limiting factor per operating point. Efficiency for the KV260 rows is decode t/s/W relative to CPU Only at the same thread count; for the STM32 it is relative to KV260 FPGA+CPU at T2[a].

Threads	Mode	Latency (ms)	<500 ms	Efficiency	Limiting factor
KV260 (4× Cortex-A53)					
T1	CPU Only	1136	No	baseline	—
T1	FPGA+CPU	613	No	+44%	Single-thread CPU throughput ceiling
T2	CPU Only	602	No	baseline	—
T2	FPGA+CPU	**452**	**Yes**	+10%	AXI burst serialization
T3	CPU Only	**424**	**Yes**	baseline	—
T3	FPGA+CPU	**488**	**Yes**	−25%	CPU parallelism offsets FPGA
T4	CPU Only	**433**	**Yes**	baseline	—
T4	FPGA+CPU	685	No	−47%	UIO interrupt contention
STM32MP257F-DK (2× Cortex-A35)					
T2	CPU Only	781	No	3.7× more efficient[a]	A35 core throughput

[a]Decode t/s/W relative to KV260 FPGA+CPU T2: 1.60 vs. 0.429 t/s/W.

Table 3.5: Design-level architectural constraints that bound the throughput ceiling independent of thread count.

Constraint	**Stage**	**Ceiling impact**
Per-burst AXI setup overhead (≈45 cycles) makes many short bursts slower than fewer large ones; a merged burst mitigates it in Stages 2a/2b but is infeasible in Stage 5	2a, 2b, 5	DDR utilization capped at 38–41% of peak
The memory scheduler cannot write to four nibble-storage banks within the same pipeline clock cycle, regardless of storage technology or partitioning strategy	5	Stage 5 is load-bound; down-projection floored at 1.62M cycles
The ZU5EV routing fabric cannot simultaneously carry the 128-bit weight reads, gate-cache lookups, and scale-factor accesses of a fully-unrolled output MAC array at 250 MHz	5	Prevents the fully-unrolled output MAC array

4 Conclusions and Future Work

4.1 Conclusions

This work presented a selective FPGA offload strategy for LFM2.5-1.2B, a model co-designed for CPU inference [4], targeting the MatMul + SwiGLU feed-forward block to enable conversational audio inference pipelines on resource-constrained edge platforms. The implementation integrates with the existing llama.cpp/ggml flow and demonstrates measurable energy-efficiency gains, especially at low and moderate thread counts, while preserving framework compatibility. Although the FPGA path does not consistently exceed CPU Only throughput at higher thread counts, the results clarify the limiting factors: practical on-device performance is jointly determined by compute parallelism, weight/data movement, and host-device orchestration. Importantly, the design already operates near the usable logic and routing budget of the target FPGA, so further gains are less likely to come from naive parallel replication and more likely from memory-path restructuring, overlap of transfers with compute, and tighter stage-level balancing (notably in Stages 2a/2b and 5). Overall, the study confirms that selective feed-forward acceleration is a viable and energy-effective step toward conversational-level edge inference, and provides a concrete roadmap for the next optimizations in LFM2.5-Audio-1.5B deployment.

4.2 Future Work

The deployed design attains 61% of the 3.6 t/s design ceiling, with the gap attributed to three quantified system-level constraints. Addressing these constraints defines the roadmap for future work.

The design exhibits memory-bound behavior, achieving 38–41% of peak DDR bandwidth at 34–35% sustained compute utilization across the projection stages. This is consistent with known bottlenecks in autoregressive decoding, where bandwidth utilization typically reaches only 29–43% even on higher-end platforms [34]. Closing this performance gap requires improved data movement strategies. Future work will therefore focus on memory-centric optimizations, including tiling, double-buffering, and layout restructuring, targeting higher sustained bandwidth utilization.

At the system level, host–accelerator overhead limits scalability. In the current design, each generated token incurs a fixed synchronous handoff: the CPU programs the accelerator's AXI-Lite control registers, issues a start signal, and blocks until a UIO interrupt is delivered, a per-token overhead that does not amortize with sequence length and grows under host-side scheduler pressure at higher thread counts. In contrast, other co-designed frameworks achieve more stable scaling through tighter compiler–runtime integration [13]. Future work will investigate asynchronous execution and pipelined scheduling to reduce synchronization overhead and improve utilization.

Extending hardware coverage beyond the MatMul + SwiGLU block to additional kernels is also a key direction for improving end-to-end performance. This will require additional compute fabric on the FPGA. Importantly, the design already effectively utilizes the available resources on the KV260, and remaining inefficiencies stem primarily from memory and system-level constraints, rather than insufficient optimization effort. Similar work pushing the same platform to its limit required complex operator fusion and custom data arrangements to maximize memory bandwidth [24].

Although the low DSP utilization (20%) might suggest that mapping the 4-bit $\times$ INT8 inner product to DSP48E2 slices could free LUT fabric, this substitution was evaluated during development and found to increase routing pressure: wider DSP fanout paths introduced additional timing-critical nets that impeded timing closure at 250 MHz. The DSP slice savings were therefore insufficient to offset the routing overhead at the current operating point, and the LUT-based multiply-accumulate path was retained.

Overall, future improvements will require co-optimization of data movement, scheduling, and numerical representation, as well as evaluation on platforms with higher bandwidth and resources, bridg-

ing the gap between lightweight selective acceleration and more general co-design frameworks.

Bibliography

[1] Youssef Abadade, Anas Temouden, Hatim Bamoumen, Nabil Benamar, Yousra Chtouki, and Abdelhakim Senhaji Hafid. A comprehensive survey on tinyml. *IEEE Access*, 11:96892–96922, 2023.

[2] Norah N. Alajlan and Dina M. Ibrahim. Tinyml: Enabling of inference deep learning models on ultra-low-power iot edge devices for ai applications. *Micromachines*, 13(6), 2022.

[3] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the April 18-20, 1967, spring joint computer conference*, volume 30 of *AFIPS '67 (Spring)*, pages 483–485, New York, NY, USA, 1967. ACM.

[4] Alexander Amini, Anna Banaszak, Harold Benoit, Arthur Böök, Tarek Dakhran, Song Duong, Alfred Eng, Fernando Fernandes, Marc Härkönen, Anne Harrington, Ramin Hasani, Saniya Karwa, Yuri Khrustalev, Maxime Labonne, Mathias Lechner, Valentine Lechner, Simon Lee, Zetian Li, Noel Loo, Jacob Marks, Edoardo Mosca, Samuel J. Paech, Paul Pak, Rom N. Parnichkun, Alex Quach, Ryan Rogers, Daniela Rus, Nayan Saxena, Bettina Schlager, Tim Seyde, Jimmy T. H. Smith, Aditya Tadimeti, and Neehal Tumma. Lfm2 technical report, 2025.

[5] Jianxin Bi, Kevin Yuchen Ma, Ce Hao, Mike Zheng Shou, and Harold Soh. Vla-touch: Enhancing vision-language-action models with dual-level tactile feedback, 2025.

[6] Longfei Chen and Yunyuan Dong. An intelligent voice interaction system based on raspberry pi and faster-whisper: Design and implementation of edge-cloud hybrid architecture. In *Proceedings of the 2025 11th Annual International Conference on Network and Information Systems for Computers*, ICNISC '25, page 61–65, New York, NY, USA, 2025. Association for Computing Machinery.

[7] Alexandre Défossez et al. Moshi: a speech-text foundation model for real-time dialogue, 2025.

[8] Stefano Di Leo, Luca De Cicco, and Saverio Mascolo. Real-time speech-to-text on edge: A prototype system for ultra-low latency communication with ai-powered nlp. *Information*, 16(8), 2025.

[9] Rina A. Doherty and Paul Sorenson. Keeping users in the flow: The role of system response time in user-perceived performance. *Procedia Manufacturing*, 3:4384–4391, 2015.

[10] Mostafa Elsharkawy, Tee Hui Teo, and I-Chyn Wei. Fpga implementation of tiny transformer using high-level-synthesis for biomedical applications. In *2025 IEEE 18th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoC)*, pages 706–709, 2025.

[11] Junhao Geng, Dongyao Jia, Zihao He, Nengkai Wu, and Ziqi Li. Enhanced conformer-based speech recognition via model fusion and adaptive decoding with dynamic rescoring. *Applied Sciences*, 14(24), 2024.

[12] Xue Geng, Zhe Wang, Chunyun Chen, Qing Xu, Kaixin Xu, Chao Jin, Manas Gupta, Xulei Yang, Zhenghua Chen, Mohamed M. Sabry Aly, Jie Lin, Min Wu, and Xiaoli Li. From algorithm to hardware: A survey on efficient and safe deployment of deep neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 36(4):5837–5857, 2025.

[13] J. Haris, R. Saha, W. Hu, and J. Cano. Designing efficient llm accelerators for edge devices, 2024.

[14] Chunming He, Yuqi Shen, Chengyu Fang, Fengyang Xiao, Longxiang Tang, Yulun Zhang, Wangmeng Zuo, Zhenhua Guo, and Xiu Li. Diffusion models in low-level vision: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 47(6):4630–4651, 2025.

[15] Soroush Heydari and Qusay H. Mahmoud. Tiny machine learning and on-device inference: A survey of applications, challenges, and future directions. *Sensors*, 25(10), 2025.

[16] Jialei Huang, Shuo Wang, Fanqi Lin, Yihang Hu, Chuan Wen, and Yang Gao. Tactile-VLA: Unlocking vision-language-action model's physical knowledge for tactile generalization, 2026. OpenReview.

[17] Xiaoyuan Huang, Hongcheng Wang, Shiyin Qin, and Su-Kit Tang. Embedded artificial intelligence: A comprehensive literature review. *Electronics*, 14(17), 2025.

[18] Yuzhe Huang, Pei Lin, Wanlin Li, Daohan Li, Jiajun Li, Jiaming Jiang, Chenxi Xiao, and Ziyuan Jiao. Taf-vla: Tactile-force alignment in vision-language-action models for force-aware manipulation, 2026.

[19] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. A survey on large language models for code generation. *ACM Trans. Softw. Eng. Methodol.*, 35(2), January 2026.

[20] Oumayma Jouini, Kaouthar Sethom, Abdallah Namoun, Nasser Aljohani, Meshari Huwaytim Alanazi, and Mohammad N. Alanazi. A survey of machine learning in edge computing: Techniques, frameworks, applications, issues, and research directions. *Technologies*, 12(6), 2024.

[21] Niks Kordjukovs and Danilo Pietro Pau. Complexity analysis for categorized edge language models. *Symmetry*, 18(5), 2026.

[22] Yunus Koç, Ahmet Ayberk Tarçın, and Doğukan Köse. Evaluation of voice recognition platforms and methods for edge ai devices. In *2024 8th International Artificial Intelligence and Data Processing Symposium (IDAP)*, pages 1–6, 2024.

[23] Bingbing Li, Santosh Pandey, Haowen Fang, Yanjun Lyv, Ji Li, Jieyang Chen, Mimi Xie, Lipeng Wan, Hang Liu, and Caiwen Ding. Ftrans: Energy-efficient acceleration of transformers using fpga, 2020.

[24] Jindong Li, Tenglong Li, Guobin Shen, Dongcheng Zhao, Qian Zhang, and Yi Zeng. Pushing up to the limit of memory bandwidth and capacity utilization for efficient llm decoding on embedded fpga, 2025.

[25] Richie Li and Sicheng Chen. Design and implementation of an fpga-based hardware accelerator for transformer, 2025.

[26] Siyuan Lu, Meiqi Wang, Shuang Liang, Jun Lin, and Zhongfeng Wang. Hardware accelerator for multi-head attention and position-wise feed-forward in the transformer, 2020.

[27] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems*, volume 5, 2023.

[28] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision, 2023.

[29] Mohaimenul Azam Khan Raiaan, Md. Saddam Hossain Mukta, Kaniz Fatema, Nur Mohammad Fahad, Sadman Sakib, Most Marufatul Jannat Mim, Jubaer Ahmad, Mohammed Eunus Ali, and Sami Azam. A review on large language models: Architectures, applications, taxonomies, open issues and challenges. *IEEE Access*, 12:26839–26874, 2024.

[30] Vijay Janapa Reddi. Generative ai at the edge: Challenges and opportunities. *Acmqueue*, 2025.

[31] STMicroelectronics. *STM32MP257 Datasheet (DS14284 Rev. 5)*, 2025.

[32] STMicroelectronics. *STM32MP257F-DK Discovery Kit User Manual (UM3385 Rev. 2)*, 2025.

[33] Xinghang Tan and Tee Hui Teo. Design and implementation of a bram-banked double-buffered matrix multiplication accelerator for transformer models on edge fpgas. In *2025 IEEE 18th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoC)*, pages 582–585, 2025.

[34] Shulin Zeng, Jun Liu, Guohao Dai, Xinhao Yang, Tianyu Fu, Hongyi Wang, Wenheng Ma, Hanbo Sun, Shiyao Li, Zixiao Huang, Yadong Dai, Jintao Li, Zehao Wang, Ruoyu Zhang, Kairui Wen, Xuefei Ning, and Yu Wang. Flightllm: Efficient large language model inference with a complete mapping flow on fpgas, 2024.